CS714: Secure Computation

Adithya Vadapalli

IIT Kanpur

February 9, 2026

A Three-Way Coin Toss

Goal

1. There are three parties P_0, P_1, and P_2.
2. They would like to securely do a three-way toss among them.

Additive Secret Shares

Shares over a Ring $\mathbb{Z}_{2^{64}}$

If P_0 holds x_0, P_1 holds x_1, s, P_n holds x_n such that $(x_1 + x_2 \ldots + x_n) \mod \mathbb{Z}_{2^{64}} = x$. Then we say each party holds additive shares of x.

XOR Secret Shares

Bitwise XOR Shares over $\mathbb{F}_2$

Let $x \in \{0,1\}^\ell$ be a bitstring. If parties $P_0, P_1, \ldots, P_n$ hold shares $x_0, x_1, \ldots, x_n \in \{0,1\}^\ell$ such that

$$x = x_0 \oplus x_1 \oplus \cdots \oplus x_n,$$

then we say they hold *XOR shares* of x.

Why This Is Sharing over a Field

XOR is addition in the finite field $\mathbb{F}_2$:

$$0 \oplus 0 = 0, \quad 0 \oplus 1 = 1, \quad 1 \oplus 1 = 0.$$

Thus, each bit of x is additively shared over $\mathbb{F}_2$. An ℓ-bit XOR sharing is equivalent to ℓ independent additive sharings in $\mathbb{F}_2$.

Example: Secret Sharing over $\mathbb{F}_8$

The Field $\mathbb{F}_8$

$\mathbb{F}_8$ is a finite field with $8 = 2^3$ elements. It can be represented as:

$$\mathbb{F}_8 = \mathbb{F}_2[\alpha]/(\alpha^3 + \alpha + 1)$$

Each field element corresponds to a 3-bit vector.

Additive Secret Sharing over $\mathbb{F}_8$

Let $x \in \mathbb{F}_8$. If parties $P_0, P_1, \ldots, P_n$ hold shares $x_0, x_1, \ldots, x_n \in \mathbb{F}_8$ such that $x = x_0 + x_1 + \cdots + x_n$ (over $\mathbb{F}_8$), then they hold additive shares of x over $\mathbb{F}_8$.

Connection to XOR Sharing

Since $\mathbb{F}_8$ is a vector space over $\mathbb{F}_2$, addition in $\mathbb{F}_8$ corresponds to bitwise XOR on 3-bit representations. Thus, sharing in $\mathbb{F}_8$ is equivalent to three correlated XOR sharings over $\mathbb{F}_2$.

Finite Fields of Size 2^k

Finite Fields

A finite field $\mathbb{F}_q$ is a set with:

- Addition and multiplication
- Additive and multiplicative inverses
- No zero divisors

Such a field exists *iff* $q = p^k$ for a prime p.

Finite Fields of Size 2^k

The Field $\mathbb{F}_8$

$$\mathbb{F}_8 = \mathbb{F}_{2^3}$$

It has 8 elements and characteristic 2. Each element can be represented as a polynomial:

$$a_0 + a_1\alpha + a_2\alpha^2 \quad (a_i \in \mathbb{F}_2)$$

with arithmetic done modulo an irreducible polynomial, e.g.

$$\alpha^3 + \alpha + 1 = 0.$$

From $\mathbb{F}_8$ to XOR Shares

Vector Space View

$\mathbb{F}_8$ is a 3-dimensional vector space over $\mathbb{F}_2$. Each element corresponds to a 3-bit vector:
$a_0 + a_1\alpha + a_2\alpha^2 \longleftrightarrow (a_0, a_1, a_2)$

Addition in $\mathbb{F}_8$

Addition is coefficient-wise over $\mathbb{F}_2$: $(a_0, a_1, a_2) + (b_0, b_1, b_2) = (a_0 \oplus b_0,\ a_1 \oplus b_1,\ a_2 \oplus b_2)$
Thus, **addition in $\mathbb{F}_8$ is bitwise XOR**.

Implication for Secret Sharing

Additive sharing over $\mathbb{F}_8$ is equivalent to

- three coupled XOR sharings over $\mathbb{F}_2$
- but with meaningful *field multiplication*

What Does $\mathbb{F}_2[\alpha]/(\alpha^3 + \alpha + 1)$ Mean?

Quotient Construction

$\mathbb{F}_2[\alpha]/(f(\alpha))$ means:

- work with polynomials over $\mathbb{F}_2$
- identify polynomials that differ by a multiple of $f(\alpha)$
- equivalently: reduce modulo $f(\alpha)$

For $\mathbb{F}_8$

Using $f(\alpha) = \alpha^3 + \alpha + 1$: $\alpha^3 = \alpha + 1$ Every element has a unique remainder: $a_0 + a_1\alpha + a_2\alpha^2$

Analogy

$\mathbb{Z}/7\mathbb{Z}$: integers mod 7 $\mathbb{F}_2[\alpha]/(f)$: polynomials mod f

What Does $\alpha^3 = \alpha + 1$ Mean?

Defining Relation

In the quotient

$$\mathbb{F}_2[\alpha]/(\alpha^3 + \alpha + 1),$$

the polynomial $\alpha^3 + \alpha + 1$ is defined to be zero.

Rewrite Rule

Thus, inside the field: $\alpha^3 = \alpha + 1$ This is not an identity, but a reduction rule.

Analogy

In $\mathbb{Z}/7\mathbb{Z}$: $7 = 0 \quad \Rightarrow \quad 8 = 1$ Same idea, different objects.

Why Finite Fields Have Size p^k

Characteristic

In any finite field F, repeated addition of 1 must cycle. The smallest p such that

$$p \cdot 1 = 0$$

is the characteristic of F.

Why p Must Be Prime

If $p = ab$, then:

$$(a \cdot 1)(b \cdot 1) = 0$$

which would create zero divisors. Hence p is prime.

Vector Space View

Every finite field F contains $\mathbb{F}_p$ and is a k-dimensional vector space over it. Thus:

$$|F| = p^k$$

A Multiplication Protocol

Why $\mathbb{Z}_{2^k}$ Is Not a Field

Recall: What a Field Requires

A field has:

- no zero divisors
- multiplicative inverses for all nonzero elements

Zero Divisors in $\mathbb{Z}_{2^k}$

In $\mathbb{Z}_{2^k}$:

$$2 \cdot 2^{k-1} = 2^k \equiv 0 \pmod{2^k}$$

but neither factor is zero.

Concrete Example

In $\mathbb{Z}_8$:

$$2 \cdot 4 = 8 \equiv 0$$

Hence, 2 and 4 are zero divisors.

What Does "No Zero Divisors" Mean?

Definition

A ring has *no zero divisors* if:

$$a \cdot b = 0 \Rightarrow a = 0 \text{ or } b = 0$$

Bad: $\mathbb{Z}_8$

$$2 \cdot 4 = 8 \equiv 0$$

with $2 \neq 0$, $4 \neq 0$. Thus, 2 and 4 are zero divisors.

Why Fields Care

Zero divisors prevent multiplicative inverses. Hence, fields cannot have zero divisors.

Why Zero Divisors Break Inverses

Assume an Inverse Exists

Let $a \neq 0$ and suppose a^{-1} exists.

Zero Divisor Assumption

If $a \cdot b = 0$ for some $b \neq 0$, then:

$$a^{-1}(a \cdot b) = 0$$

$$(a^{-1}a)b = b = 0$$

Contradiction.

Conclusion

An element with a multiplicative inverse cannot be a zero divisor.

Protocol CEPS

Circuit Evaluation with Passive Security

The protocol evaluates an arithmetic circuit securely against *passive* adversaries.

- Parties: $P_1, \ldots, P_n$
- Field: $\mathbb{F}$
- Adversary threshold: $t < n/2$

Protocol CEPS

Circuit Evaluation with Passive Security

The protocol evaluates an arithmetic circuit securely against *passive* adversaries.

- Parties: $P_1, \ldots, P_n$
- Field: $\mathbb{F}$
- Adversary threshold: $t < n/2$

Three Phases

1. Input sharing
2. Computation
3. Output reconstruction

Input Sharing Phase

- Each party P_i holds input $x_i \in \mathbb{F}$
- P_i distributes a t-degree sharing

$$[x_i; f_{x_i}]_t$$

  where $f_{x_i}(0) = x_i$
- Input gates are now considered processed

Key Invariant

Invariant

Let $a \in \mathbb{F}$ be the value assigned to any wire whose gate has been processed.

Then the players hold a sharing

$$[a; f_a]_t$$

for some degree-t polynomial f_a with $f_a(0) = a$.

Key Invariant

Invariant

Let $a \in \mathbb{F}$ be the value assigned to any wire whose gate has been processed.

Then the players hold a sharing

$$[a; f_a]_t$$

for some degree-t polynomial f_a with $f_a(0) = a$.

- Invariant holds after input sharing
- Protocol ensures it holds after every gate

Computation Phase

- Process gates in computational (topological) order
- For each unprocessed gate:
  - Inputs already satisfy the invariant
  - Locally compute output sharing

Addition Gate

- Inputs: $[a; f_a]_t$, $[b; f_b]_t$
- Players compute locally:

$$[a; f_a]_t + [b; f_b]_t = [a + b; f_a + f_b]_t$$

- Degree remains t
- Invariant preserved

Multiply-by-Constant Gate

- Input: $[a; f_a]_t$
- Constant: $\alpha \in \mathbb{F}$
- Players compute:

$$\alpha[a; f_a]_t = [\alpha a; \alpha f_a]_t$$

- Degree unchanged
- Invariant preserved

Multiplication Gate: Problem

- Inputs: $[a; f_a]_t$, $[b; f_b]_t$
- Naive multiplication gives:

$$f_a \cdot f_b$$

- Degree becomes $2t$
- Violates invariant

Multiplication Gate: Problem

- Inputs: $[a; f_a]_t$, $[b; f_b]_t$
- Naive multiplication gives:

$$f_a \cdot f_b$$

- Degree becomes $2t$
- Violates invariant

Goal

Reduce degree back to t while preserving the value ab

Multiplication Gate: Degree Reduction

1. Players compute
$$[a; f_a]_t \cdot [b; f_b]_t = [ab; f_a f_b]_{2t}$$
2. Define $h = f_a f_b$
$$h(0) = ab$$
3. Each P_i holds $h(i)$ and reshapes it into
$$[h(i); f_i]_t$$

Multiplication Gate: Recombination

- $\deg(h) = 2t \leq n - 1$
- Let $r = (r_1, \ldots, r_n)$ be recombination vector

$$h(0) = \sum_{i=1}^{n} r_i h(i)$$

- Players compute:

$$\sum_i r_i [h(i); f_i]_t = [h(0); \sum_i r_i f_i]_t = [ab; f_{ab}]_t$$

Computation Phase Summary

- All gates processed
- Invariant preserved throughout
- Every wire value is held as a t-sharing

Output Reconstruction

- For each output gate i, players hold:

$$[y_i; f_{y_i}]_t$$

- Each P_j sends $f_{y_i}(j)$ to P_i
- P_i reconstructs:

$$y_i = f_{y_i}(0)$$

using Lagrange interpolation from any $t + 1$ shares

Protocol CEPS: Circuit Evaluation with Passive Security

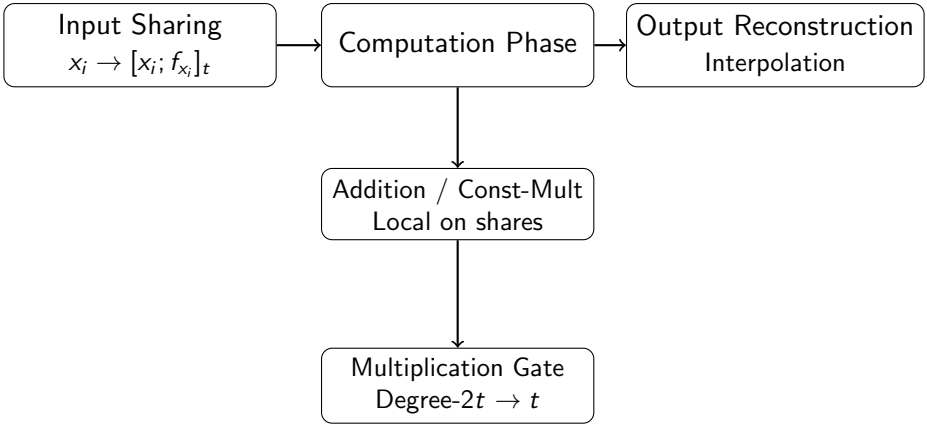
Setting:

- Parties evaluate an arithmetic circuit over a field $\mathbb{F}$
- Security against *passive (honest-but-curious)* adversaries

Protocol Phases:

1. **Input Sharing:**
   - Each input x_i is secret-shared as $[x_i; f_{x_i}]_t$
   - All input gates are considered processed
2. **Computation Phase:**
   - *Invariant:* Every processed wire with value a is held as $[a; f_a]_t$
   - Addition / constant-multiplication: done locally
   - Multiplication:
     - Multiply shares $\Rightarrow$ degree-$2t$ polynomial
     - Degree reduction via recombination vector $\mathbf{r}$
     - Resulting sharing preserves value ab
3. **Output Reconstruction:**
   - Parties send shares to output player
   - Output recovered via Lagrange interpolation

CEPS Protocol Overview (Passive Security)



Invariant: Every processed wire with value a is held as a sharing $[a; f_a]_t$.

What Remains to Prove

- **Correctness:** Outputs equal circuit evaluation
- **Privacy:** Corrupted parties learn nothing beyond their inputs and outputs

What Remains to Prove

- **Correctness:** Outputs equal circuit evaluation
- **Privacy:** Corrupted parties learn nothing beyond their inputs and outputs

Security Model

Passive (honest-but-curious) adversary corrupting at most $t < n/2$ parties

Correctness

- Recall the invariant:

  Every processed wire with value a is held as $[a; f_a]_t$

- After all gates are processed:
  - Each output wire holds $[y_i; f_{y_i}]_t$
  - $f_{y_i}(0) = y_i$
- Output reconstruction returns exactly y_i

Correctness

- Recall the invariant:

  Every processed wire with value a is held as $[a; f_a]_t$

- After all gates are processed:
  - Each output wire holds $[y_i; f_{y_i}]_t$
  - $f_{y_i}(0) = y_i$
- Output reconstruction returns exactly y_i

Conclusion

Protocol CEPS computes the correct circuit output

What Do Corrupted Parties See?

Corrupted parties receive only two types of messages from honest parties:

Type 1: During Computation

- Shares from input sharing
- Shares sent during multiplication degree-reduction

Type 2: During Output Reconstruction

- Shares of output polynomials $f_{y_i}(j)$

Observation 1: Computation Phase Privacy

- Honest parties' messages are shares of random degree-t polynomials
- Corrupted set has size at most t
- Therefore:
  - They see at most t points on a degree-t polynomial
  - Distribution is information-theoretically uniform

Observation 1: Computation Phase Privacy

- Honest parties' messages are shares of random degree-t polynomials
- Corrupted set has size at most t
- Therefore:
  - They see at most t points on a degree-t polynomial
  - Distribution is information-theoretically uniform

Conclusion

Messages during input sharing and multiplication reveal nothing

Observation 2: Output Reconstruction Privacy

- Corrupted parties already know:
  - Their own shares of f_{y_i}
  - The output value $y_i = f_{y_i}(0)$ (if corrupted)
- Reconstruction gives them $t + 1$ points on f_{y_i}

Observation 2: Output Reconstruction Privacy

- Corrupted parties already know:
  - Their own shares of f_{y_i}
  - The output value $y_i = f_{y_i}(0)$ (if corrupted)
- Reconstruction gives them $t + 1$ points on f_{y_i}

Key Point

Since $\deg(f_{y_i}) = t$, these values are fully determined by inputs and outputs already known to corrupted parties

Privacy Intuition

- Computation phase: only random-looking shares
- Output phase: values corrupted parties could compute themselves

Privacy Intuition

- Computation phase: only random-looking shares
- Output phase: values corrupted parties could compute themselves

Crucial Insight

Nothing beyond inputs and outputs is ever leaked

Simulation Paradigm

Goal

Construct a simulator S that reproduces the corrupted parties' view using:

- Inputs of corrupted parties
- Outputs of corrupted parties

Simulator Construction (High Level)

The simulator S proceeds as follows:

1. Run corrupted parties internally on their inputs
2. Simulate honest-party messages by sampling uniformly random shares
3. In output reconstruction:
   - Ensure consistency with known outputs

Simulator Construction (High Level)

The simulator S proceeds as follows:

1. Run corrupted parties internally on their inputs
2. Simulate honest-party messages by sampling uniformly random shares
3. In output reconstruction:
   - Ensure consistency with known outputs

Result

Simulated view is identically distributed to real execution

Formalizing Observation 1

To formalize privacy, define a stripping function:

Strip(view$_j$)

Removes from party P_j's view:

- Shares received from honest parties in output reconstruction

Leaves:

- Random coins
- Messages during computation

Lemma 3.4 (Privacy)

Lemma

Let $C \subseteq \{P_1, \ldots, P_n\}$ be corrupted parties, $|C| \leq t$.
Let $x^{(0)}, x^{(1)}$ be two global inputs such that:

$$x_i^{(0)} = x_i^{(1)} \quad \forall i \in C$$

Then:

$$\{\text{Strip}(\text{view}_j^{(0)})\}_{j \in C} \equiv \{\text{Strip}(\text{view}_j^{(1)})\}_{j \in C}$$

Meaning of the Lemma

- Corrupted parties cannot distinguish executions
- As long as their own inputs are unchanged
- Regardless of honest parties' inputs

Meaning of the Lemma

- Corrupted parties cannot distinguish executions
- As long as their own inputs are unchanged
- Regardless of honest parties' inputs

Conclusion

Protocol CEPS is perfectly private

Big Picture

- CEPS = BGW-style MPC for arithmetic circuits
- Privacy via:
  - Polynomial secret sharing
  - Degree bounds
  - Simulation-based reasoning
- Blueprint for active security extensions

CS714: Secure Computation

Adithya Vadapalli

IIT Kanpur

February 9, 2026

Setting

- Parties $P_1, \ldots, P_n$ jointly compute $y = f(x_1, \ldots, x_n)$.
- Adversary is *passive*.
- Let $C \subseteq \{P_1, \ldots, P_n\}$ be corrupted parties.

Views

- View of party P_i:

$$\text{view}_i = (\text{input}, \text{randomness}, \text{messages})$$

- Joint corrupted view:

$$\text{view}_C = \{\text{view}_i\}_{i \in C}$$

Output Reconstruction

- Output y is secret-shared.
- Corrupted parties see only their output shares.
- Honest shares are never revealed.

Observation 2 (Informal)

Observation

Once the output y is fixed, the distribution of corrupted parties' views does not depend on honest parties' inputs.

Formalizing Observation 2

- Define a deterministic transformation:

$$\mathsf{Dress}_C(y, \mathsf{view}_C)$$

- Removes dependence on honest parties' randomness.
- Produces a canonical representative of the view.

Goal of Simulation

- Build a simulator $\mathcal{S}$ such that:

$$\text{view}_C \approx \mathcal{S}(x_C, y)$$

- Simulator only knows corrupted inputs and output.

Simulator Strategy

1. Sample random honest shares consistent with y.
2. Generate protocol messages to corrupted parties.
3. Locally simulate corrupted-party computation.
4. Apply Dress_C.

Why Simulation Works

- Secret sharing is information-theoretic.
- Honest randomness is perfectly hidden.
- Output reconstruction is the only coupling point.

Passive Security Theorem

Theorem

Protocol CEPS securely evaluates any arithmetic circuit with passive security against any corrupted set C.

Multiplication Setup

- Values $a = f(0)$, $b = g(0)$ are shared using degree-1 polynomials.
- Example:
$$f(x) = 2 + x, \quad g(x) = 3 - x$$
- Goal: compute $c = ab$.

Local Multiplication

- Each party computes:
$$d_i = f(i) \cdot g(i)$$
- Points lie on:
$$d(x) = f(x)g(x)$$
- Degree increases to 2.

Resulting Polynomial

$$d(x) = (2 + x)(3 - x) = 6 + x - x^2$$

- Constant term gives correct output.
- Degree growth must be handled.

Degree Reduction

- Parties reshare d_i using fresh randomness.
- Linear recombination removes high-degree terms.
- Example:

$$c = 3d_1 - 3d_2 + d_3$$

Security of Multiplication

- Fresh randomness masks intermediate values.
- Corrupted parties see uniform shares.
- Only c is revealed.

Why the Bound Matters

- Security proven for $|C| \leq t$.
- What if $|C| > t$?

High-Level Intuition

- Too many shares allow interpolation.
- Honest inputs become reconstructible.
- Privacy is information-theoretically impossible.

Optimality Claim

Theorem

If more than t parties are corrupted, no perfectly private protocol exists.

Two-Party Setting

- Parties P_1, P_2 with inputs $b_1, b_2 \in \{0, 1\}$.
- Compute Boolean function $f(b_1, b_2)$.

Assumptions

Perfect Correctness

Output is always $f(b_1, b_2)$.

Perfect Privacy

Each party's view depends only on its input and the output.

Transcript Sets

- Let $T(b_1, b_2)$ be the set of possible transcripts.
- Privacy implies overlaps.
- Correctness forbids overlaps across outputs.

AND Function Example

$$f(b_1, b_2) = b_1 \wedge b_2$$

- Privacy:

$$T(0,0) \cap T(0,1) \neq \emptyset$$

- Privacy:

$$T(0,1) \cap T(1,1) \neq \emptyset$$

Contradiction

- Implies:

$$T(0,0) \cap T(1,1) \neq \emptyset$$

- But correctness requires disjointness.
- Contradiction.

Impossibility Result

Theorem

No two-party protocol can compute a nontrivial Boolean function with perfect correctness and perfect privacy.

Big Picture

- Simulation formalizes privacy.
- Multiplication is the core technical challenge.
- Corruption bounds are tight.
- Two parties are fundamentally insufficient.